Digital Status

Implementation Roadmap

A phase-by-phase guide to building the career-vault and portfolio-rental platform: accounts, templates, recruiter gate, billing, super admin, and custom domains.

Stack: Express + better-sqlite3 (backend), React + Vite (frontend)
Version 1.0 - October 2026

1. Objectives

Every decision in this project must serve at least one of these objectives. Before building any feature, check which objective it serves. If none, postpone it.

1. **Stop people rewriting documents.** Everything is stored once and shared by link or domain.
2. **The owner controls their data.** Visitor contacts and messages are visible only to the portfolio owner, never to the super admin.
3. **Recruiter gate.** Email OTP gives a preview; a contact form unlocks the download.
4. **Paid tiers** with real, enforced feature differences.
5. **Super admin** handles support, resets, invitations, and platform metrics only.

2. Key Decisions Up Front

Decision	Choice	Why
Public URL at first	yourplatform.com/u/slug	Custom domains are the hardest part, so they come last
Sessions	HttpOnly cookie	Safer than localStorage
Validation	zod on every endpoint	Stops bad data and injection early
Migrations	Numbered SQL files (001_users.sql, ...)	Safe schema changes
Private data	Encrypted fields (AES-256-GCM)	Keeps the 'super admin cannot see it' promise
Payments	Sandbox first, then MTN MoMo / Airtel via a provider	Matches the Rwanda market
Feature limits	plans and plan_features tables	No hardcoded if (tier === ...)

Folder structure

```
backend/
  src/
    config/        env, db, mail, storage
    migrations/    001_..., 002_...
    middleware/    auth, role, planLimit, rateLimit, validate
    modules/
      auth/  portfolios/  templates/  documents/
      public/  recruiter/  leads/  billing/  admin/
    utils/         crypto, tokens, sanitize
  private-documents/   (never served statically)
frontend/src/
  pages/       public/ dashboard/ admin/ auth/
  templates/   (one folder per template + registry.js)
  components/  api/  hooks/
```

3. The Phases

Phase 0: Foundation (2 to 3 days)

- Migration runner; run `PRAGMA foreign_keys = ON` and `journal_mode = WAL`.
- helmet, cors (locked to FRONTEND_URL), express-rate-limit, central error handler, zod.
- Make the port-in-use message in server.js Windows-friendly.
- Confirm .env is in .gitignore.

Done when: the server starts, migrations apply, and a /health route works.

Phase 1: Accounts and Auth (1 week)

- Tables: users (with role: customer or super_admin), sessions, email_verifications, password_resets.
- Endpoints: POST /auth/signup, /verify-email, /login, /logout, /forgot-password, /reset-password, GET /me.
- Hash passwords with argon2 or bcrypt, rate-limit login, invalidate sessions on password change.
- Seed the super admin from .env.
- Frontend: login, signup, verify, reset pages and a protected-route wrapper.

Done when: a new user can sign up, verify email, log in, and reset a password; a customer gets 403 on any admin route.

Phase 2: Portfolio Core and Editor (1 to 1.5 weeks)

- Tables: portfolios (owner, slug, template_id, theme_json, status draft/published), portfolio_sections (type, order, data_json, is_private).
- Endpoints: CRUD for portfolio and sections, PUT /portfolios/:id/publish, slug availability check.
- Editor UI: a form per section (Hero, About, Experience, Education, Projects, Skills, Certificates, Contact), live preview, save.
- Sanitize all text and allow only https:// URLs.

Done when: a user can build a full profile and preview it; drafts are invisible to the public.

Phase 3: Templates (1.5 weeks)

See the Template Guide in section 4.

Done when: switching templates re-renders the same data with no data loss.

Phase 4: Documents and Storage (1 week)

- Table: documents (owner, type, original_name, stored_name, size, mime, is_primary_cv).
- Upload with multer: allow only pdf/png/jpg, enforce size limits, use random stored names.
- Store files in private-documents/ and never serve the folder statically.
- Endpoints: upload, list, delete, replace CV, owner-only download.
- Enforce plan limits (count and MB) via middleware.

Done when: a user can upload and replace a CV, and a direct URL guess returns 404.

Phase 5: Public Page and Recruiter Gate (1.5 weeks)

This is the heart of the product. Build it carefully.

1. `GET /public/:slug` returns **only public sections**. Nothing private is in the response.
2. `POST /recruiter/request-otp` takes an email, creates a hashed 6-digit code that expires in 10 minutes, and rate-limits per IP and per email.
3. `POST /recruiter/verify-otp` allows 5 attempts then locks. On success it sets a short-lived HttpOnly cookie.
4. `GET /public/:slug/full` returns private sections, **server-side only after verification**.
5. `POST /recruiter/contact` takes name, email, position, phone, reason. Encrypt the fields, store in recruiter_contacts with owner_user_id, and unlock the download.
6. `GET /public/:slug/download` streams the CV via a short-lived signed link.
7. Email the owner: 'Someone downloaded your CV.'

Done when: page source and network responses before OTP show no private data, and the download fails without a completed form.

Phase 6: Owner Dashboard (1 week)

- Leads list (decrypted only for the owner), inquiries inbox, view and download counts.
- Export leads as CSV and export all of the user's data.
- Share-link box with copy button and QR code.
- Public contact form saving to inquiries (encrypted).

Done when: an owner sees every visitor and message, and a test confirms a different owner or an admin gets 403 on those endpoints.

MVP milestone: after Phase 6 you have a usable product. Test it with 5 to 10 real people before building billing.

Phase 7: Plans and Billing (1.5 weeks)

- Tables: plans, plan_features, subscriptions, payments.
- requireFeature('otp_gate') middleware and checkLimit('documents') helper, both reading plan_features.
- Pricing page, checkout flow, and a webhook route that verifies signatures and is idempotent.
- The server computes the price. Never trust the client.
- node-cron job: move subscriptions to a 14-day grace period, then pause the page. Data is kept and export always works.
- Renewal reminders by email.

Done when: a free user hits a limit and sees an upgrade prompt, and an expired plan pauses the page without deleting anything.

Phase 8: Super Admin (1 week)

- Dashboard of counts and revenue only.
- User list (account info only), suspend/reactivate, password-reset trigger that emails a link to the user.
- Single-use, expiring invitation links for people who contact you before signing up.
- Support tickets, abuse reports, template and plan management.
- admin_audit_logs for every admin action, visible to the affected user.

Done when: automated tests prove no admin route returns recruiter contacts, inquiries, or documents.

Phase 9: Custom Domains and Production (1.5 to 2 weeks)

- Table: custom_domains (portfolio_id, domain, verification_token, status, verified_at).
- The user adds a domain; you give them a DNS TXT record to prove ownership; only then activate it.
- Host-header middleware resolves the domain to a portfolio and rejects unknown hosts.
- Use **Caddy** with on-demand TLS (much easier than Nginx for customer domains). Allow issuance only for verified domains via an 'ask' endpoint.
- Deploy to a VPS; run with pm2; back up SQLite with VACUUM INTO plus off-server copies; add logging and uptime monitoring.

Done when: a real domain pointed at your server serves the right portfolio over HTTPS.

Phase 10: Hardening and Launch (1 week)

- Terms of Service and Privacy Policy (cover Rwanda's data-protection law).
- SPF, DKIM, DMARC and a transactional email provider.
- Security checklist pass: XSS, CSP, rate limits, upload tests.
- Report-abuse flow, restore-tested backups, error tracking.

4. Template Guide

A template is a **React component that reads data**, not a hand-written page. This is what lets you make new templates quickly.

1. Common data contract

Every template receives the same props:

```
{ profile: { name, headline, photo, bio, contact },
  sections: [ { type: 'experience', data: {...} }, ... ],
  theme: { primary, background, font, radius },
  mode: 'public' | 'preview',
  gate: { verified: boolean } }
```

2. Shared section components

Build these once and reuse everywhere: Hero, About, Experience, Education, Projects, Skills, Certificates, VideoPitch, Contact, and DownloadCV (the gate button). A template then only decides **layout and style**.

3. Theme tokens

Use CSS variables (--primary, --bg, --font, --radius). Users change colors and fonts through validated tokens (hex check and font allowlist), **never raw CSS**. Allow raw CSS only on Enterprise, after sanitizing.

4. Registry

```
// templates/registry.js
export const templates = {
  minimal_grid: { component: MinimalGrid, tier: 'free' },
  clean_resume: { component: CleanResume, tier: 'free' },
  tech_pro:     { component: TechPro,     tier: 'pro'  },
  creative:     { component: Creative,    tier: 'pro'  },
};
```

The public page does `const T = templates[portfolio.template_id].component`. The server also checks the tier on save, so a free user cannot select a paid template by tampering with the request.

5. Build order

#	Template	Tier	Layout
1	minimal_grid	Free	Single column, cards. Build first and finish all shared sections with it.
2	clean_resume	Free	Two-column CV-style layout.
3	tech_pro	Pro	Dark theme, project cards, skills grid.
4	creative	Pro	Large hero, video pitch, animated timeline.

Do not start the paid templates until the first one is fully working. Two good templates beat ten rough ones.

6. Template checklist

- Mobile-first (most recruiters open links on phones).
- Handles empty sections gracefully (hide them).
- Recruiter gate overlay is a shared component, not copied into each template.
- Semantic HTML, alt text, good contrast.
- Prints cleanly (@media print) as a bonus.
- Fast public rendering: lazy-load images and video.

5. Reference

Core tables at a glance

users, sessions, portfolios, portfolio_sections, documents, recruiter_otps, recruiter_sessions, recruiter_contacts (encrypted), inquiries (encrypted), plans, plan_features, subscriptions, payments, custom_domains, invitations, support_tickets, admin_audit_logs.

Super admin boundaries

Allowed	Not allowed
Password-reset trigger (link emailed to the user)	Viewing visitor contacts, inquiries, or documents
Support tickets, suspending abusive accounts	Entering a customer account without a recorded, consented impersonation
Platform metrics, templates, plans, failed payments	Reading any encrypted owner data
Single-use invitation links	Acting without an audit-log entry

Rough timeline (solo, part-time)

Stage	Phases	Duration
MVP (usable, free)	0 to 6	about 7 to 9 weeks
Revenue	7 and 8	about 2.5 weeks
Custom domains and launch	9 and 10	about 3 weeks

Rules to build faster

1. Finish one phase completely (including its 'done when' test) before the next.
2. Write security tests as you go, especially 'admin cannot read leads' and 'private data absent before OTP'.
3. Ship the free tier first to real users and let their feedback shape pricing.
4. Do not build analytics maps, calendar booking, or multi-domain Enterprise features until you have paying users.
5. Keep a TODO.md with these phases as checkboxes.